

Quest Log — Build Tasks for Claude Code

Give these to your agent **one task at a time, in order**. Don't skip ahead — each task assumes the previous one is done and working. After each task, run/test before moving to the next.

TASK 1 — Project scaffold

Set up a new Vite + React + Tailwind project named `quest-log`. Initialize a git repo. Install and configure Tailwind CSS so a test className like `bg-blue-500` renders correctly on the default page. Add a `.env` file with placeholders `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`, and add `.env` to `.gitignore`. **Done when:** `npm run dev` shows a styled Tailwind test page.

TASK 2 — Supabase schema

I have a Supabase project with URL and anon key in `.env`. Write the SQL to create these 7 tables with Row Level Security enabled (policy: `user_id = auth.uid()` on all except content is static, no table needed for that):

```
users(id uuid pk references auth.users, start_date date,
daily_logs(id uuid pk default gen_random_uuid(), user_id
weight_logs(id uuid pk default gen_random_uuid(), user_id
tiers(user_id uuid references users pk, current_tier int
trials(id uuid pk default gen_random_uuid(), user_id uuid
xp_stats(user_id uuid references users pk, strength_xp in
achievements(id uuid pk default gen_random_uuid(), user_i
```

Output this as a single `.sql` file I can paste into the Supabase SQL editor. Also create `src/lib/supabaseClient.js` that initializes the Supabase JS client from the env vars. **Done when:** the SQL file exists and the client file connects without errors.

TASK 3 — Auth + onboarding flow

Build a sign-up/login flow using Supabase Auth (email magic link). After first login, if no row exists in `users` for this account, show an onboarding form with these fields in this exact order:

1. Height (cm), starting weight (kg or lb — pick one, note it), age
 2. A 7-day multi-select labeled "Which days are you free to train?" (Mon–Sun checkboxes)
 3. Protein target input, default value 160, editable On submit: auto-pick one of the selected days as `run_day` (the one closest to the middle of the spread of selected days), insert a row into `users` with `start_date = today`, and insert starter rows into `tiers` (`current_tier=1`, `tier_started_at=today`) and `xp_stats` (all zero). **Done when:** a new account can sign up, complete onboarding, and lands on an empty "Today" page with their data saved in Supabase.
-

TASK 4 — Static content files

Create these files under `src/content/` as plain JS exports (arrays/objects, no database):

- `workoutPools.js` — 6 tiers (index 0-5), each an array of 4-5 objects `{key, name, desc, xp}` for bodyweight strength quests. Tier 0 = wall push-ups/towel rows/bodyweight squats/plank (beginner). Tier 5 = full push-ups/pull-up tests/endurance squats (advanced). Progress logically between.
- `runProgression.js` — 12 objects (one per week), each `{name, desc, xp}`, progressing from walk/jog intervals (1 min jog / 2 min walk) to a continuous 20-25 min run by week 12.

- `restDayQuests.js` — 5-6 objects `{key, name, desc, xp}` for 5-10 minute options: stretching, mobility work, short walk, breathing exercise.
 - `mealSidePools.js` — 6 tiers, each an array of 4-5 short challenge strings. Tier 0 = awareness (log every meal, add a vegetable). Tier 5 = precision (hit a calorie range, meal prep 3 days ahead).
 - `trials.js` — 6 objects `{tier, name, criteria, xpReward}`, one performance test per tier transition (e.g. tier 0→1: "hold plank 30 sec, 12 clean bodyweight squats, 20-min walk").
 - `achievements.js` — 8-10 objects `{key, name, condition}` describing unlock conditions in plain text (first push-up quest done, 7-day streak, first Trial cleared, etc.) **Done when:** all 6 files exist, export valid arrays, and import without errors.
-

TASK 5 — Quest generation logic (pure functions, unit tested)

Create `src/lib/questLogic.js` with these pure functions:

- `isTrainingDay(dateStr, freeDays)` → boolean, checks if the date's weekday is in the user's `free_days` array
 - `dailyHash(dateStr, salt)` → deterministic number from a string (simple char-code hash), used to seed "random" choices so the same day always gives the same quest
 - `getWorkoutQuest(dateStr, tier, isRunDay, weekIndex)` → returns a quest object from `runProgression` if `isRunDay`, otherwise a hashed pick from `workoutPools[tier]`
 - `getRestQuest(dateStr)` → hashed pick from `restDayQuests`
 - `getMealSideQuest(dateStr, tier)` → hashed pick from `mealSidePools[tier]` Write a small test file (Vitest) confirming: same date+tier always returns the same quest; different tiers return quests from different pools; `isTrainingDay` correctly filters by weekday. **Done when:** `npm run test` passes for this file.
-

TASK 6 — Today screen (core loop)

Build the Today page. On load: check if today's row exists in `daily_logs` for this user — if not, create it using `isTrainingDay` + the quest generation functions from Task 5 to populate `workout_quest_key` or `rest_quest_key`, and `meal_side_quest_key`. Render:

- If training day: the workout quest card (name, description, XP, "Mark complete" button)
 - If rest day: the rest quest card instead, same layout, labeled as optional
 - Meal quest card: protein number input, the day's side-quest text, "Mark complete" button On marking complete: update the `daily_logs` row, and add the quest's XP to the correct field in `xp_stats` (workout quest → `strength_xp` or `endurance_xp` depending on whether it was a run or strength quest; rest quest → `discipline_xp`; meal quest → `nutrition_xp`). **Done when:** completing a quest updates Supabase and persists across a page refresh.
-

TASK 7 — Streak and multiplier

Add `computeStreak(userId)` to `questLogic.js` (or a new `streakLogic.js`) — walks backward day by day from today through `daily_logs`, counting a day as "complete" if: training day → `workout_done` AND `meal_quest_done`; rest day → `meal_quest_done` only. Stop counting at the first incomplete day (today is exempt from breaking the streak if not yet acted on). Add `computeMultiplier(streak)` returning `Math.min(0.5, streak * 0.02)`. Display current streak, best streak (track separately, don't recompute historically each time — store `best_streak` somewhere sensible, e.g. a new column on `users`), and the multiplier percentage on the Today screen. Apply the multiplier to XP awarded in Task 6's completion handler. **Done when:** the streak number updates correctly across multiple simulated days (test by manually adjusting `daily_logs` dates in Supabase).

TASK 8 — Character screen

Build a Character page. Fetch `xp_stats`, sum all four stats into one total XP, compute an overall level using an increasing threshold formula (e.g. level N requires $100 + (N-1) * 40$ cumulative XP). Map level ranges to Rank letters (E: 1-9, D: 10-19, C: 20-29, B: 30-39, A: 40-49, S: 50+). Render: rank badge, level number, an XP progress bar to next level, and four individual stat bars (Strength/Endurance/Discipline/Nutrition) each showing their own XP total. **Done when:** the page reflects real XP values from Supabase and updates after completing quests on Today.

TASK 9 — Weight tracking

On the Today screen, check if 7+ days have passed since the last `weight_logs` entry for this user (or none exists) — if so, show a small weigh-in prompt card with a number input and "Log weight" button that inserts into `weight_logs`. Build a Progress page with a line chart (use Recharts) plotting `weight_logs` over time, smoothed with a simple rolling average (e.g. 3-entry moving average) rather than raw points. **Done when:** a weigh-in can be logged and appears on the chart.

TASK 10 — 12-week grid

On the Progress page, add a grid: 12 rows (weeks) × 7 boxes (days), starting from `start_date`. For each past/current day, color the box based on `daily_logs`: both required quests done = filled, one done = half-filled, neither = empty/miss. Future days render as an outline only. Fetch all `daily_logs` rows for the date range in a single query. **Done when:** the grid renders correctly and matches the actual completion history.

TASK 11 — Trial system

Add logic that checks: if `today - tier_started_at >= 14 days` and no pending trial exists for the current tier, insert a new row into `trials` (status `pending`) using the matching entry from `content/trials.js`. Show a Trial card on the Today screen when one is pending — the test's name/criteria and two buttons, "Clear it" and "Not yet." On "Clear it": update `tiers.current_tier += 1`, reset `tier_started_at = today`, mark the trial `cleared_at = today`, status `cleared`. On "Not yet": leave the tier as-is, mark status `retry`, and don't generate a new trial until another 7 days pass. **Done when:** clearing a trial actually changes which tier's quest pool Task 5's functions pull from — verify a new (harder) quest appears the next day.

TASK 12 — Achievements

Add `checkAchievements(userId)` that runs after any quest completion in Task 6 — checks simple conditions against `daily_logs/trials/streak` data (first workout quest ever completed, 7-day streak reached, first trial cleared, etc. — use the list in `content/achievements.js`) and inserts newly unlocked ones into the `achievements` table, avoiding duplicates. Show a brief toast/banner when a new achievement unlocks. List all unlocked achievements permanently on the Character page. **Done when:** completing the right actions unlocks the correct achievements exactly once.

TASK 13 — Settings + polish

Build a Settings page: edit `free_days`, `protein_target`, and a "Reset all data" button (with confirmation) that clears the user's rows across all tables except `users` itself. Add loading and empty states to every page built so far. Do a mobile-width responsive pass on all screens (this is a phone-first app). **Done when:** every screen works cleanly on a 375px-wide viewport and settings changes persist.

TASK 14 — PWA setup

Add a web app manifest and service worker (use `vite-plugin-pwa`) so the site is installable via "Add to Home Screen" on both iOS Safari and Android Chrome. Add basic app icons. **Done when:** the installed icon launches the app in standalone mode (no browser chrome) on a test phone.

TASK 15 — Deploy

Connect the repo to Vercel, set the two Supabase env vars in the Vercel project settings, and confirm a production deploy works end to end (sign up → onboarding → complete a quest → refresh → data persists). **Done when:** a fresh visitor on the live URL can go through the full flow with no errors.